llm-diff

Product Specification • v1.0

A CLI tool for comparing LLM outputs — semantically, visually, and at scale.

Status: Spec	Type: CLI Tool	Platforms: Mac · Linux · Win	Lang: Python + TS

1. Overview

Developers who work with LLMs are constantly asking the same question: did my change make things better or worse? Switching models, tweaking prompts, or adjusting temperature all produce subtly different outputs — and there is no fast, standard way to compare them.

llm-diff is a command-line tool that compares two LLM responses and surfaces the differences clearly: word-level, semantic, or structural. It works across any two OpenAI-compatible API endpoints, supports batch comparisons, and can export shareable HTML reports.

> Think of it as `git diff` for language model outputs — but diff-aware of meaning, not just characters.

The core problem

- Model outputs are long, prose-heavy, and hard to compare manually.
- Developers waste significant time copy-pasting responses into text editors or spreadsheets.
- No standard OSS tool handles this — existing solutions are either vendor-locked or built for single-turn prompt testing, not comparison.
- Teams switching between models (GPT-4o vs Claude 3.5 vs Mistral) have no quick way to validate the tradeoffs.

Who it's for

Primary users

- AI engineers testing prompt changes
- Developers benchmarking model upgrades
- Teams evaluating vendor switches (e.g., OpenAI → Anthropic)
- Researchers comparing model behavior

Secondary users

- Product managers reviewing AI output quality
- QA engineers building regression checks
- Open-source contributors to LLM projects

- Technical writers evaluating summarization quality

2. Core Features

2.1 Three diff modes

The tool's central feature — choose the mode that matches what you care about:

Mode	What it shows	Best for
`--word`	Word-level additions and deletions, color-coded in terminal	Prompt tweaks, copy changes
`--semantic`	Meaning-level differences using embedding similarity scoring	Model comparisons, behavior drift
`--json`	Machine-readable diff with scores, changed segments, and metadata	CI pipelines, programmatic use

2.2 CLI interface

The primary interface. Designed to be instantly learnable — a developer should be productive within 60 seconds of installation.

```
# Basic comparison — two models, same prompt
$ llm-diff "Explain recursion" --a gpt-4o --b claude-3-5-sonnet

# Compare two prompt files against the same model
$ llm-diff --prompt-a v1.txt --prompt-b v2.txt --model gpt-4o

# Semantic diff with score
$ llm-diff "Summarize this doc" --a gpt-4o --b mistral-large --semantic
  Similarity: 78%  |  Key divergences: tone, structure, 3rd paragraph

# Batch compare from a YAML prompt file
$ llm-diff --batch prompts.yml --a gpt-4o --b gpt-4-turbo --out
report.html
```

2.3 Prompt file support

Users can define a prompt.yml file to run structured, repeatable comparisons without retyping prompts.

```
# prompts.yml
prompts:
  - id: summarize
    text: "Summarize the following in 3 sentences: {input}"
    inputs: [sample1.txt, sample2.txt]
  - id: rewrite
```

```
    text: "Rewrite this in a formal tone: {input}"
    inputs: [sample3.txt]
```

2.4 HTML report output

Export a self-contained HTML file showing the diff side-by-side, with color coding, semantic scores, and metadata. No server required — works fully offline and can be shared via email or Slack.

- Side-by-side view with highlighted additions, deletions, and changes
- Semantic similarity score per paragraph
- Prompt metadata: model, temperature, token count, latency
- No external dependencies — single HTML file, embeds all CSS/JS inline

2.5 Multi-provider support

Works with any OpenAI-compatible API endpoint out of the box. Providers are configured via environment variables or a .llmdiff config file.

- OpenAI (GPT-4o, GPT-4-turbo, o1, etc.)
- Anthropic (Claude 3.5 Sonnet, Claude 3 Opus, etc.)
- Groq, Mistral, Together AI, Perplexity
- Local models via Ollama or LM Studio (localhost endpoints)
- Any custom endpoint following the OpenAI messages API format

3. Full CLI Reference

Flag	Short	Type	Default	Description
`--prompt`	`-p`	`string`	—	The prompt text to send to both models
`--prompt-a`	—	`path`	—	Path to a text file used as prompt for model A
`--prompt-b`	—	`path`	—	Path to a text file used as prompt for model B
`--model-a`	`-a`	`string`	—	Model identifier for side A (e.g. gpt-4o)
`--model-b`	`-b`	`string`	—	Model identifier for side B (e.g. claude-3-5-sonnet)
`--model`	`-m`	`string`	—	Use the same model for both sides (prompt diff mode)
`--mode`	—	`enum`	word	Diff mode: word | semantic | json
`--semantic`	`-s`	`flag`	false	Shorthand for --mode semantic
`--json`	`-j`	`flag`	false	Output raw JSON diff to stdout
`--batch`	—	`path`	—	Path to a prompts.yml file for batch comparison
`--out`	`-o`	`path`	—	Save HTML report to this path

`--save`	—	`flag`	false	Auto-save HTML report to ./diffs/ directory
`--temperature`	`-t`	`float`	0.7	Temperature passed to both models
`--max-tokens`	—	`int`	1024	Max tokens for each model's response
`--timeout`	—	`int`	30	Request timeout in seconds
`--no-color`	—	`flag`	false	Disable terminal color output
`--verbose`	`-v`	`flag`	false	Show full API request/response metadata
`--version`	—	`flag`	—	Print version and exit

4. Output Specification

4.1 Terminal output (default)

Color-coded, readable in any terminal. Follows standard diff conventions adapted for prose.

```
────────────────────────────────────────────────────────────────────────────────
  llm-diff  •  gpt-4o  vs  claude-3-5-sonnet
  Prompt: "Explain recursion in simple terms"
────────────────────────────────────────────────────────────────────────────────


  [A] gpt-4o
  Recursion is when a function calls itself to solve
- a smaller version of the same problem.

  [B] claude-3-5-sonnet
  Recursion is when a function calls itself to solve
+ a smaller piece of the same problem, until it
+ reaches a base case that stops the repetition.

  Similarity: 84%  |  Tokens: 61 / 73  |  Latency: 0.9s / 1.2s
```

4.2 JSON output schema

Used for scripting, CI pipelines, or feeding into other tools.

```
{
  "prompt": "Explain recursion in simple terms",
  "model_a": "gpt-4o",
  "model_b": "claude-3-5-sonnet",
  "similarity_score": 0.84,
  "tokens": { "a": 61, "b": 73 },
  "latency_ms": { "a": 920, "b": 1210 },
  "diff": [
    { "type": "equal", "text": "Recursion is when..." },
    { "type": "delete", "text": "a smaller version" },
    { "type": "insert", "text": "a smaller piece...base case" }
```

```
  ]
}
```

5. Technical Architecture

5.1 Python package (primary)

- **Entry point:** `llm_diff/cli.py` (Click-based CLI)
- **Diff engine:** `difflib` for word-level; `sentence-transformers` for semantic mode
- **API layer:** `openai` SDK (compatible with all providers via base_url override)
- **Terminal rendering:** `rich` for color, layout, and tables
- **HTML report:** `Jinja2` template, vanilla JS, no external deps
- **Config:** `.llmdiff` TOML file in project root or home dir

5.2 TypeScript package (secondary — npm)

A thinner wrapper for JS/TS developers. Shares the same JSON output schema as the Python package. Enables usage in Node scripts, CI pipelines, and VS Code extensions.

- Published to npm as llm-diff
- Programmatic API: import { diff } from 'llm-diff'
- No Python dependency required — standalone implementation

5.3 Project structure

```
llm-diff/
├── llm_diff/
│   ├── cli.py          # Click entry point
│   ├── diff.py         # Core diff engine
│   ├── semantic.py     # Embedding-based scoring
│   ├── providers.py    # API abstraction layer
│   ├── report.py       # HTML report generator
│   └── config.py       # .llmdiff config loader
├── templates/
│   └── report.html.j2  # Jinja2 report template
├── tests/
│   ├── test_diff.py
│   └── test_providers.py
├── pyproject.toml
└── README.md
```

6. Non-Functional Requirements

Performance

Compatibility

- Cold-start (no API call): < 200ms
- Word diff on 2,000-word outputs: < 100ms
- Semantic diff (embedding): < 2s (local model)
- Concurrent API calls for A and B — never sequential

Reliability

- Retry with backoff on API rate limit errors
- Graceful failure with clear error messages
- Timeout handling with partial result display

- Python 3.9+ required
- Node 18+ for the npm package
- macOS, Linux, Windows (WSL tested)
- Works in VS Code terminal, iTerm2, standard shells

Security

- API keys never logged or stored
- Keys read from env vars or .env file only
- No telemetry without explicit opt-in
- HTML reports contain no external requests

7. Release Roadmap

Phase	Milestone	Duration	Success Metric
v0.1 — MVP	Core diff engine, two-model CLI comparison, colored terminal output	2 weeks	Works end-to-end on any two OpenAI-compatible endpoints
v0.2 — Modes	Semantic diff mode, word-level diff mode, JSON output, --save flag	2 weeks	3 diff modes working reliably
v0.3 — Prompt history	prompt.yml support, batch mode, prompt file diffing	1 week	Can diff 10 prompts in one command
v0.4 — Dashboard	HTML report output, shareable diff artifacts	1 week	Report loads offline with no dependencies
v1.0 — Stable release	Full docs, test suite, PyPI + npm publish, README with demos	1 week	100 GitHub stars within 2 weeks of launch

> Scope discipline: resist adding features before v1.0 ships. The goal of v0.1 is to be useful to one real developer. Everything else follows from that.

8. Success Metrics

Measured at v1.0 launch and 30 days post-launch:

Metric	Target

GitHub stars at launch week	**100+**
Time to first successful diff (new user)	**< 60 seconds**
CLI cold-start latency (no API call)	**< 200ms**
PyPI / npm downloads in month 1	**500+**
Issues opened in first 2 weeks (signals real usage)	**10+**
Discord / community members at v1.0	**50+**

9. Out of Scope for v1.0

> These are deliberate exclusions. Adding them before v1.0 risks shipping nothing.

- GUI or web application (this is a CLI tool first)
- Fine-tuning or training integrations
- Image, audio, or multimodal output diffing
- Saved diff history or database persistence
- Team accounts or authentication
- Agent or multi-turn conversation diffing (v2 candidate)
- Automated prompt optimization suggestions

10. Open Questions

These need answers before or during v0.2:

❓ **Semantic diff model**
sentence-transformers/all-MiniLM-L6-v2 is fast and local, but requires a ~80MB download on first run. Is that acceptable? Alternative: use an embedding API call instead.

❓ **Config file format**
TOML (.llmdiff) vs YAML vs .env only. TOML is cleanest but adds one dependency.

❓ **Provider auth UX**
Should the tool auto-detect provider from model name (e.g. 'claude-*' → Anthropic key) or require explicit --provider flag?

❓ **Semantic scoring method**
Cosine similarity of full response embeddings vs sentence-level scoring averaged. Sentence-level is more precise but slower.

❓ **npm package scope**
Publish as llm-diff (if available) or @yourname/llm-diff? Check npm availability before launch.